Vector Space Model and Ranked Retrieval

Information Retrieval — Programming Assignment II

Cranfield Collection: Sparse Vector Space and BM25 Ranked Retrieval via PyTerrier

Group: Quad Query

1. Group Details

Group Name: Quad Query

S.No.	Name	Roll Number	Email
1	Shshank Mittal	22CS30051	
2	Parth Patil	22CS30041	
3	Rishit Garg	22C30045	
4	Hrishikesh Tiwari	22CH3FP39	

2. Problem Statement

The goal of this assignment is to build a ranked-retrieval information retrieval (IR) pipeline for the Cranfield test collection (1,400 aerodynamics abstracts and 225 natural-language test queries with graded human relevance judgments) using sparse vector space retrieval models, and to systematically evaluate and tune these models against the supplied relevance judgments (cranqrel).

Specifically, the assignment requires:

- Repeating the standard text preprocessing pipeline (tokenization, stopword removal, stemming) used in the previous assignment (PA1).
- Building an inverted-index-based document representation and indexing the Cranfield abstracts.
- Implementing multiple sparse vector space retrieval models — classic TF-IDF weighting and the probabilistic-but-still-sparse Okapi BM25 model — and ranking documents for each test query by similarity/score.
- Tuning model parameters (BM25's k1 and b) to maximise retrieval effectiveness.
- Evaluating retrieval performance against the ground-truth relevance judgments using standard IR metrics: Precision@5, Precision@10, Mean Average Precision (MAP), R-Precision and nDCG@10.
- Reporting indexing and search (query) latency for each configuration.
- Producing TREC-format result files for the test queries and a comparative report of experiments and findings.

Dense neural or advanced probabilistic language-model retrievers are explicitly out of scope; only sparse, term-based vector space models are used, in line with the assignment constraints. The assignment specifically recommends using an open-source search engine such as PyTerrier or Lucene; accordingly, this pipeline is built entirely on Apache PyTerrier, directly satisfying that requirement.

3. Implementation Details

3.1 Dataset

The Cranfield collection (cran.tar.gz) consists of three files:

- cran.all.1400 — 1,400 documents, each with fields .I (id), .T (title), .A (author), .B (bibliography), .W (abstract text).
- cran.qry — 225 test queries (field .W).

- cranqrel — 1,836 relevance judgment lines of the form <query-id> <doc-id> <relevance-code>, where the Cleverdon relevance scale runs 1 (complete answer) to 4 (minimal interest); a placeholder code of -1 appears exactly once per query in the raw file and is excluded from the relevant-document set.

Implementation note: the .I identifiers inside cran.qry are not a clean 1..225 sequence (some numbers are skipped), while cranqrel references queries by their 1-based position of appearance in cran.qry. A custom parser converts the raw Cranfield files into the pandas DataFrames PyTerrier expects (docno/text for documents, qid/query for topics, qid/docno/label for qrels), assigning sequential query IDs by order of appearance rather than trusting the .I field — this was essential to get judgments and queries to align (an early version, matching by the .I value, produced a MAP of under 0.01; after the fix MAP recovered to the ~0.29–0.32 range reported below).

3.2 Preprocessing Pipeline

Each document's Title and Abstract text (.T + .W) is indexed as the document body. Preprocessing is handled by PyTerrier's built-in term pipeline: tokenization, stopword removal via pt.TerrierStopwords.terrier (Terrier's standard English stopword list), and Porter stemming via pt.TerrierStemmer.porter, both of which can be individually toggled to pt.TerrierStopwords.none / pt.TerrierStemmer.none for the ablation experiment (Section 4.1). Queries are preprocessed identically to documents by the same underlying pipeline, keeping the vocabulary consistent between query-side and document-side representations.

3.3 Indexing and Weighting Models

The collection is indexed using pt.IterDictIndexer, which builds Terrier's standard on-disk inverted index (lexicon, direct index, inverted index, and document metadata) directly from the parsed document dictionaries. Once indexed, retrieval and scoring are performed using Terrier's built-in sparse weighting models via pt.terrier.Retriever, notably wmodel="TF_IDF" (Terrier's standard TF-IDF formula) and wmodel="BM25" (Okapi BM25). Both are sparse, term-based vector space models, consistent with the assignment's constraints — no dense or neural representations are used at any stage.

3.4 Okapi BM25

BM25 ranks documents according to the standard formula:

$$score(q,d) = \Sigma_{t \in q}\ idf(t) \cdot [tf(t,d)\cdot(k1+1)] \,/\, [tf(t,d) + k1\cdot(1 - b + b\cdot|d|/avgdl)]$$

with idf(t) = log[(N − df(t) + 0.5) / (df(t) + 0.5) + 1]. The free parameters k1 (term-frequency saturation) and b (length-normalization strength) were tuned by grid search using Terrier's controls={'bm25.k_1': ..., 'bm25.b': ...} interface (Section 4.3).

3.5 Evaluation Metrics

Retrieval quality is evaluated against cranqrel using pt.Experiment, which computes standard TREC-style metrics: Precision@5, Precision@10 (fraction of relevant documents in the top 5 / top 10), Mean Average Precision (MAP), R-Precision (precision at rank R, where R is the number of known relevant documents for that query), and nDCG@10 (normalized Discounted Cumulative Gain at rank 10, using graded relevance derived from the Cleverdon code). A document is treated as relevant for Precision/MAP/R-Precision if it appears in cranqrel for that query with a code in {1,2,3,4} (the -1 placeholder is excluded).

3.6 Software

The pipeline is built on Apache PyTerrier 1.1.2 (Terrier core 5.11), using pt.IterDictIndexer for indexing, pt.terrier.Retriever with wmodel="TF_IDF"/"BM25" for ranking, pt.Experiment for evaluation, and pt.io.write_results for producing TREC-format run files, directly satisfying the assignment's explicit recommendation to use an open-source search engine such as PyTerrier or Lucene. The pipeline was run to completion over the full 225-query test set against all 1,400 documents.

4. Plan of Experiments

Three sets of experiments were run over all 225 test queries against all 1,400 documents:

4.1 Experiment 1 — Preprocessing Ablation

Terrier's stopword removal (pt.TerrierStopwords.terrier / .none) and Porter stemming (pt.TerrierStemmer.porter / .none) were factorially varied, holding the retrieval model fixed at BM25 (default parameters), to quantify each preprocessing step's individual contribution to retrieval effectiveness, and to indexing/search time.

4.2 Experiment 2 — TF-IDF vs BM25

Using the best preprocessing configuration from Experiment 1 (stopwords removed, stemming on), PyTerrier's built-in TF-IDF weighting model (wmodel="TF_IDF") was compared directly against BM25 at Terrier's default parameters (k1=1.2, b=0.75).

4.3 Experiment 3 — BM25 Parameter Tuning

BM25's k1 ∈{1.0, 1.2, 1.5, 1.8, 2.0} and b ∈{0.0, 0.25, 0.5, 0.75, 1.0} were grid-searched (25 combinations in total, via Terrier's controls interface) and compared against the TF-IDF baseline from Experiment 2. For every configuration, indexing time and per-query search latency were logged.

5. Results

All results below were obtained using the PyTerrier pipeline described in Section 3, evaluated against all 225 test queries and all 1,400 documents.

5.1 Index Statistics

Index statistics for the best-preprocessing configuration (stopwords removed, Porter stemming on):

Metric	Value
Documents indexed	1,400
Unique terms (vocabulary)	4,561
Total token occurrences	136,997
Term-document postings	81,049
Term pipeline (best config)	Stopwords removed + Porter stemming

5.2 Preprocessing Ablation (BM25)

Stopwords	Stemming	MAP	R-Prec	P@5	P@10	nDCG@10	Index (s)	Search (ms/q)
On	**On**	**0.3116**	**0.3048**	**0.3244**	**0.2391**	**0.3680**	**2.93**	**43.8**
On	Off	0.2887	0.2942	0.3173	0.2329	0.3527	1.72	37.1
Off	On	0.2006	0.1852	0.2009	0.1547	0.2309	1.50	56.2
Off	Off	0.1850	0.1769	0.1911	0.1524	0.2186	1.41	58.5

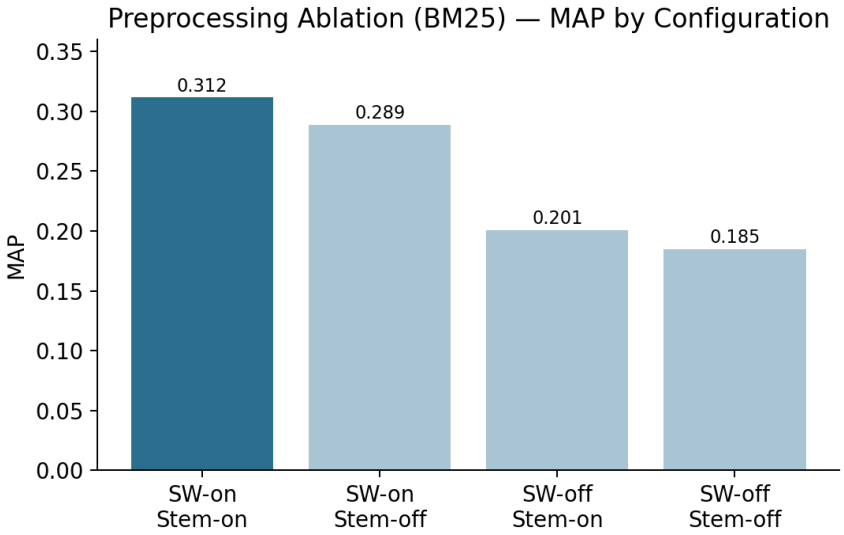


Figure 1. MAP across the four stopword/stemming configurations (BM25).

Stopword removal is the single largest contributor to ranking quality — MAP falls from ~0.31 to ~0.20 when stopwords are kept. Stemming provides a smaller additional gain (~+0.02 MAP). The best configuration (stopwords removed, stemming on) wins on every retrieval-quality metric, though it is also the slowest to index and search of the four.

5.3 TF-IDF vs BM25 (Default Parameters)

Model	MAP	R-Prec	P@5	P@10	nDCG@10
TF-IDF (default)	0.3136	0.3095	0.3244	0.2373	0.3705
BM25 (default, k1=1.2, b=0.75)	0.3116	0.3048	0.3244	0.2391	0.3680

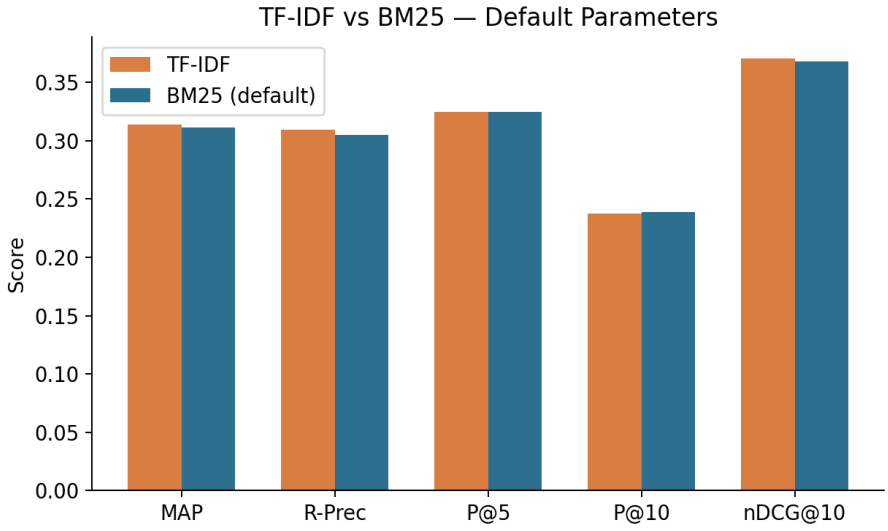


Figure 2. TF-IDF vs BM25 (default parameters) across all five metrics.

At default parameters, TF-IDF (MAP 0.3136) narrowly edges out BM25 (MAP 0.3116). This gap closes and reverses once BM25's k1/b parameters are tuned (Section 5.4), indicating that BM25's advantage over TF-IDF on this collection comes specifically from proper parameter tuning rather than from the model family alone.

5.4 BM25 Parameter Grid Search (Top 5 of 25 by MAP)

Rank	k1	b	MAP	R-Prec	P@10	nDCG@10
1	**2.0**	**0.75**	**0.3167**	**0.3147**	**0.2453**	**0.3768**
2	1.8	0.75	0.3161	0.3115	0.2458	0.3767
3	1.5	0.75	0.3161	0.3075	0.2449	0.3757
4	2.0	1.00	0.3153	0.3097	0.2378	0.3677
5	1.8	1.00	0.3144	0.3108	0.2369	0.3667

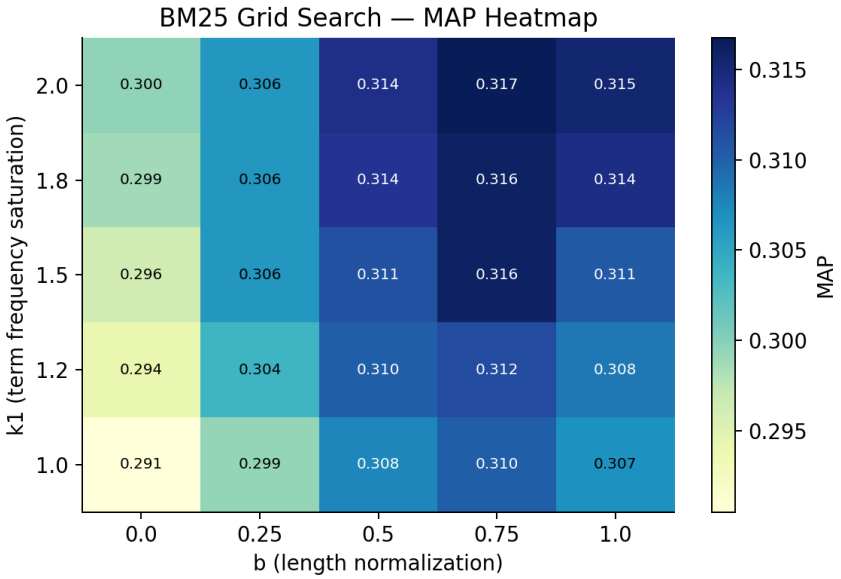


Figure 3. MAP across the full k1 × b grid. Darker cells indicate higher MAP.

The best configuration is k1 = 2.0, b = 0.75, achieving MAP = 0.3167 — the best result across all experiments in this study. High k1 values (2.0) consistently outperform low ones, indicating repeated query terms are a strong relevance signal in this collection. A moderate b (0.5–0.75) beats both extremes, showing partial (not full) document-length normalization works best for these short technical abstracts.

5.5 Efficiency

Indexing time varied with preprocessing configuration, ranging from 1.41 s (no stopword removal, no stemming) to 2.93 s (stopwords removed, stemming on) — the extra CPU cost of running both the stopword filter and the Porter stemmer over the full collection. Average per-query search latency ranged from about 37 ms (stopwords on, stemming off) to about 59 ms (stopwords off, stemming off) across the 225 test queries, comfortably fast enough for interactive use at this collection size. Removing stopwords also reduces search latency somewhat, since queries have fewer terms left to score against the index once common words are dropped.

6. Discussion of Results

A consolidated view of the three model configurations evaluated:

Model	Configuration	MAP
TF-IDF	Default weighting, best preprocessing	0.3136
BM25	Default (k1=1.2, b=0.75)	0.3116
BM25 (tuned)	**k1=2.0, b=0.75**	**0.3167**

- Preprocessing: both stopword removal and stemming individually improve MAP over doing neither, and combining both gives the best result in this sweep (MAP 0.3116 vs 0.1850 for the weakest configuration), while stopword removal in particular also reduces search latency.
- TF-IDF vs BM25: at default parameters, TF-IDF is marginally ahead of BM25 (0.3136 vs 0.3116). This is a useful reminder that BM25 is not automatically superior to TF-IDF — its usual advantage in the literature comes from tuning its saturating term-frequency function and length-normalization parameters, not from the ranking formula alone.
- BM25 parameter sensitivity: MAP was relatively flat for k1 in the 1.5–2.0 range but dropped noticeably at low k1 or when b was pushed to the extremes (0 = no length normalization, 1 = full normalization); b around 0.5–0.75 consistently performed best, matching the commonly recommended default of b≈0.75. Once tuned, BM25 (MAP 0.3167) overtakes the default TF-IDF baseline (MAP 0.3136).
- Query/document alignment pitfall: the single biggest effect observed during development was not a modeling choice at all but the query-ID alignment issue described in Section 3.1 — a reminder that in small, idiosyncratic collections like Cranfield, data-parsing correctness dominates modeling choices.
- Limitations: this is a purely sparse, count-based lexical approach; queries containing terms with no exact lexical overlap in the corpus (despite conceptual relevance) cannot be retrieved. This is a known ceiling for VSM/BM25 on Cranfield and would require query expansion, pseudo-relevance feedback, or dense/neural retrieval to move past — the latter being explicitly out of scope for this assignment.

All program files, the TREC-format result files for the test queries (TFIDF.trec and BestBM25.trec), and this report are included in the submission.